

Loan Approval Prediction System

Project Report

Submitted By: Neha Chiluka

Branch: B.Tech, CSE

College: Kakatiya Institute of Technology and Science, Warangal

Roll Number: B24CS195L

1. Objectives

- Automate loan approval decisions using machine learning, reducing reliance on manual, subjective underwriting.
- Build an end-to-end pipeline: data ingestion, preprocessing, feature engineering, model training, evaluation, and deployment.
- Compare multiple classification algorithms and select the one offering the best balance of accuracy and reliability.
- Deliver predictions through a real-time, interactive interface suitable for demonstration and practical use.
- Achieve a target classification accuracy of 80–90% on unseen test data.
-

2. Methodology

The project follows a standard supervised machine learning workflow: raw applicant data is cleaned and transformed into a consistent numeric feature space, several classification models are trained and tuned on this data, and the best-performing model is evaluated on a held-out test set before being deployed behind a real-time prediction interface. Class imbalance between approved and rejected applications is corrected using SMOTE prior to training, and model selection is driven by cross-validated F1-score to ensure balanced performance across both classes rather than only overall accuracy.

3. Algorithm

3.1 Step-by-Step Procedure

Step 1: Data Collection — Load the loan application dataset containing applicant details and the corresponding loan status.

Step 2: Data Cleaning — Handle missing values in fields such as credit history, loan amount, and selfemployment status through imputation.

Step 3: Data Encoding — Convert categorical variables (gender, marital status, education, property area, etc.) into numerical form using label or one-hot encoding.

Step 4: Feature Engineering — Derive new features: total household income, debt-to-income ratio, and logtransformed loan amount.

Step 5: Feature Scaling — Standardize numerical features so no single feature dominates due to scale.

Step 6: Train-Test Split — Divide the processed dataset into training and testing subsets using stratified sampling.

Step 7: Handling Class Imbalance — Apply SMOTE on the training set to balance approved and rejected classes.

Step 8: Model Training — Train Logistic Regression, Random Forest, and XGBoost on the balanced training data.

Step 9: Hyperparameter Tuning — Use grid search with 5-fold cross-validation to select the best configuration for each model.

Step 10: Model Evaluation — Evaluate all models on the test set using accuracy, precision, recall, F1-score, and ROC-AUC; select the best model.

Step 11: Deployment — Serialize the final model and serve it through a Streamlit interface for real-time predictions.

3.2 Model / Technique Applied

Three classification techniques are trained and compared: Logistic Regression, an interpretable linear baseline; Random Forest, an ensemble of decision trees that captures non-linear relationships and feature interactions; and XGBoost, a gradient boosting technique that builds trees sequentially to correct prior errors. The best-performing model, selected via cross-validated F1-score, is used in the final deployed system.

3.3 Process Flow

- Input — Applicant details entered via form or uploaded as a record.

- Preprocessing — Cleaning, encoding, scaling, and feature engineering applied.
- Model Inference — Processed feature vector passed to the trained classifier.
- Prediction — Model outputs an approval or rejection decision with a probability score.
- Output — Result displayed to the user through the Streamlit interface.



4. Dataset

4.1 Source

The Loan Prediction Problem Dataset, publicly available on Kaggle, was used as the data source. It contains historical loan application records with the corresponding approval decision for each applicant.

4.2 Records and Format

The dataset contains 614 applicant records, provided in CSV format, with a mix of categorical and numerical fields. Several columns (Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, Credit_History) contain missing values that were imputed prior to modeling.

4.3 Features

Feature	Description	Type
Gender	Applicant's gender	Categorical
Married	Marital status	Categorical
Dependents	Number of dependents	Categorical/Numeric
Education	Graduate / Not Graduate	Categorical
Self_Employed	Employment type	Categorical
ApplicantIncome	Monthly income of applicant	Numeric
CoapplicantIncome	Monthly income of co-applicant	Numeric

LoanAmount	Requested loan amount (in thousands)	Numeric
Loan_Amount_Term	Repayment term (in months)	Numeric
Credit_History	Meets credit guidelines (0/1)	Binary
Property_Area	Urban / Semiurban / Rural	Categorical
Loan_Status	Target: Approved (Y) / Rejected (N)	Binary

4.4 Train / Test Split

The dataset was split 80% for training (491 records) and 20% for testing (123 records), using stratified sampling to preserve the same approve-to-reject ratio in both subsets. SMOTE was applied only to the training set, bringing its class balance to 50/50, while the test set was left untouched to give an unbiased evaluation.

5. Implementation

The pipeline was implemented in Python using pandas and NumPy for data handling, scikit-learn for preprocessing and modeling (Logistic Regression, Random Forest), XGBoost for gradient boosting, and imbalanced-learn for SMOTE. Hyperparameters for all three models were tuned using GridSearchCV with 5fold stratified cross-validation. The best model, along with the fitted scaler and label encoders, was serialized using Joblib.

5.1 Code Architecture

The implementation is split across two files with distinct responsibilities:

- `Loan_Approval_Source_Code.ipynb` — the training pipeline. Loads and cleans the data, engineers features, trains and tunes all three models, evaluates them, and exports the winning model plus its preprocessing artifacts (scaler, encoders, feature column list) to the `models/` directory.
- `app.py` — the inference/serving layer. Loads the exported artifacts once at startup, accepts applicant details through a Streamlit form, applies the identical preprocessing steps used in training, and returns a live prediction with a confidence score.

Keeping training and serving as separate files, connected only through the serialized artifacts in `models/`, means the model can be retrained and swapped in without touching the interface code at all.

5.2 Key Implementation Details

Handling class imbalance — SMOTE is applied only to the training split, never to the test set, so evaluation reflects the real-world class distribution rather than an artificially balanced one:

```
smote = SMOTE(random_state=RANDOM_STATE) X_train_bal,  
y_train_bal = smote.fit_resample(X_train, y_train)
```

Model comparison — all three classifiers are tuned and evaluated through the same loop, so results are directly comparable:

```
models = {  
    "Logistic Regression": lr_search.best_estimator_,  
    "Random Forest": rf_search.best_estimator_,  
    "XGBoost": xgb_search.best_estimator_,  
} for name, model in models.items():    metrics,  
preds = evaluate(model, name)
```

Consistent preprocessing at inference time — app.py mirrors the exact encoding, feature engineering, and scaling steps used during training, so a new applicant's raw form input is transformed identically to how the training data was:

```
df["TotalIncome"] = df["ApplicantIncome"] + df["CoapplicantIncome"] df["DebtToIncomeRatio"]  
= df["LoanAmount"] / (df["TotalIncome"] / 1000 + 1e-3)  
df[numeric_cols] = scaler.transform(df[numeric_cols])  
pred = model.predict(df)[0] proba = model.predict_proba(df)[0][pred]
```

6. Results

All three models were evaluated on the held-out 123-record test set. Results are summarized below:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8455	0.8587	0.9294	0.8927	0.8712
Random Forest	0.8455	0.8929	0.8824	0.8876	0.8560
XGBoost (Best)	0.8537	0.8764	0.9176	0.8966	0.8553

XGBoost achieved the highest F1-score (0.8966) and was selected as the final deployed model, with a test accuracy of 85.37% — within the 80–90% target range set in the project's abstract. Its detailed classification report on the test set is as follows:

- Rejected class — Precision: 0.79, Recall: 0.71, F1-score: 0.75 (38 test cases)
- Approved class — Precision: 0.88, Recall: 0.92, F1-score: 0.90 (85 test cases)
- Confusion Matrix: $\begin{bmatrix} 27 & 11 \\ 7 & 78 \end{bmatrix}$ (rows = actual, columns = predicted; order: Rejected, Approved)

6.1 Discussion

XGBoost was chosen over Logistic Regression and Random Forest primarily because it achieved the highest F1-score, meaning it offered the best balance between precision and recall rather than excelling at only one. Logistic Regression had the highest recall (0.9294) but lower precision, meaning it approved more true positives but also let through more risky applicants; Random Forest had the highest precision (0.8929) but missed more good applicants. XGBoost's combination — precision 0.8764 and recall 0.9176 — is the more balanced trade-off for a lending context, where both false approvals (financial risk to the lender) and false rejections (lost business, unfair denial) carry real costs.

Looking at the confusion matrix directly: of 38 applicants who were actually rejected, the model correctly flagged 27 (71% recall on the rejected class) but let 11 slip through as false approvals. Of 85 applicants who were actually approved, it correctly approved 78 (92% recall) and incorrectly rejected 7. In a real deployment, the 11 false approvals are the more consequential error type, since they represent loans that may default — a bank using this system would likely want to pair the model's prediction with a minimum confidence threshold or a secondary manual review step for borderline cases, rather than auto-approving on the model's decision alone.

6.2 Live Prediction Demo

The screenshot below shows the deployed Streamlit interface returning a real-time prediction for a sample applicant.

Deploy ⋮

Gender	Applicant Income (monthly)
Male ▾	5000 - +
Married	Co-applicant Income (monthly)
Yes ▾	0 - +
Dependents	Loan Amount (in thousands)
0 ▾	120 - +
Education	Loan Term (months)
Graduate ▾	360 ▾
Self Employed	Credit History Meets Guidelines?
Yes ▾	Yes ▾
Property Area	
Urban ▾	

Predict

✔ Loan Approved — Confidence: 63.3%

7. Conclusion

This project demonstrates a complete, working machine learning pipeline for automating loan approval decisions. Starting from a public benchmark dataset, the system cleans and engineers relevant features, corrects for class imbalance, trains and tunes three classification models, and deploys the best-performing one (XGBoost, 85.37% test accuracy) behind a real-time Streamlit interface. The results meet the project's original target accuracy range and show a reasonable balance between precision and recall on both the approved and rejected classes. Beyond the classroom setting, this kind of system has direct real-world value for banks and NBFCs — speeding up loan processing, reducing inconsistency in manual underwriting, and supporting fairer, data-driven lending decisions. Future work could extend the system with explainability tools (e.g., SHAP) to make individual predictions more transparent to loan officers and applicants alike.